

My Boss Is an AI - Game Design Document v0.1

Vision

Create a humorous, meme-driven 2D pixel-art iPhone game. The player is an office worker. Every in-game day they choose whether to complete tasks themselves or delegate them to AI. Every choice visibly transforms the office and, eventually, the world. The tone is comedy, not fear. The player is never locked into one path: they can always shift back toward human work or further into automation.

Core pillars

1. Funny first. Every minute should contain a visual joke, animation or sound gag. 2. Tiny sessions (10-15 minutes total campaign, replayable). 3. Consequences are immediately visible. 4. Beautiful pixel art with lively animations despite simple mechanics. 5. Simple enough for a solo developer. No backend.

Gameplay loop

Start a workday, receive 3-5 office tasks, choose Human or AI for each task, watch immediate consequences, finish the day, observe how the office evolves, repeat until one of several endings.

Transformation examples

Use AI: colleague disappears after layoffs; robot cleaner appears; AI coffee machine replaces barista; manager becomes an algorithm; coworkers become bots; eventually entire city reflects automation. Choose Human more often: office stays messy but lively, coworkers interact, creativity increases. Mixed choices create hybrid worlds and unique endings.

Art direction

Cute pixel art, exaggerated facial expressions, squash-and-stretch animations, meme references, lots of environmental storytelling, funny sound effects for every interaction.

Technical direction

Swift, SwiftUI, SpriteKit preferred. Data-driven events. Local save only. No backend. Architecture should remain modular and testable.

Roadmap

Milestone 1: playable prototype. Milestone 2: office evolution. Milestone 3: endings. Milestone 4: polish, animations, sounds. Milestone 5: App Store launch.

Instructions for Claude Code

Act as Lead Game Designer, Technical Architect and Senior iOS Engineer. Challenge ideas when appropriate, but optimise for shipping quickly. Prefer maintainable code over clever code. Whenever implementing a feature, explain the reasoning first, then produce production-ready code. Keep a running TODO list and propose the next milestone automatically after each completed task.